

Candlesticks

Historical Financial Time Series for ANSI Common Lisp

`https:
//github.com/cgore/candlesticks/`

Christopher Mark Gore
`http://www.cgore.com`
`cgore@cgore.com`

Wednesday, August 26th, AD 2026

Contents

Copyright	vii
Introduction	ix
1 The Schema	1
1.1 Conventions	1
1.2 How the Tables Fit Together	2
1.3 Overview	4
1.4 The <code>instrument_types</code> Table	4
1.5 The <code>data_source_kinds</code> Table	5
1.6 The <code>exchanges</code> Table	6
1.7 The <code>instruments</code> Table	6
1.8 The <code>data_sources</code> Table	7
1.9 The <code>tickers</code> Table	7
1.10 The <code>candlestick_durations</code> Table	8
1.11 The <code>data_retrievals</code> Table	9
1.12 The <code>candlesticks</code> Table	9
1.13 What Is Not a Table	10
2 The candlesticks/config Package	11
2.1 Variables	11
2.1.1 The <code>*default-connection-spec*</code> Variable	11
2.2 Functions	11
2.2.1 The <code>default-connection-spec</code> Function	11
2.2.2 The <code>set-default-connection-spec!</code> Function	11
2.2.3 The <code>connection-spec</code> Function	12
2.2.4 The <code>current-connection</code> Function	12
2.2.5 The <code>connected-p</code> Function	12
2.2.6 The <code>require-connection!</code> Function	12
2.2.7 The <code>universal-time->timestampz</code> Function	12
2.2.8 The <code>canonicalize-slug</code> Function	12
2.2.9 The <code>to-db</code> Function	13
2.2.10 The <code>from-db</code> Function	13
2.2.11 The <code>coerce-price</code> Function	13

2.3	Macros	13
2.3.1	The with-connection Macro	13
2.3.2	The first-row Macro	14
3	The candlesticks/durations Package	15
3.1	Types	15
3.1.1	The duration Type	15
3.2	Functions	15
3.2.1	The standard-durations Function	15
3.2.2	The make-duration Function	15
3.2.3	The duration-by-short-name Function	16
3.2.4	The duration-by-long-name Function	16
3.2.5	The all-durations Function	16
3.2.6	The ensure-standard-durations Function	16
4	The candlesticks/instrument-types Package	17
4.1	Types	17
4.1.1	The instrument-type Type	17
4.2	Functions	17
4.2.1	The standard-instrument-types Function	17
4.2.2	The make-instrument-type Function	18
4.2.3	The instrument-type-by-name Function	18
4.2.4	The instrument-type-id-for Function	18
4.2.5	The all-instrument-types Function	18
4.2.6	The ensure-standard-instrument-types Function	18
5	The candlesticks/instruments Package	19
5.1	Types	19
5.1.1	The instrument Type	19
5.2	Functions	19
5.2.1	The make-instrument Function	19
5.2.2	The fill-instrument Function	20
5.2.3	The instrument-by-id Function	20
5.2.4	The all-instruments Function	20
6	The candlesticks/data-source-kinds Package	21
6.1	Types	21
6.1.1	The data-source-kind Type	21
6.2	Functions	21
6.2.1	The standard-data-source-kinds Function	21
6.2.2	The make-data-source-kind Function	21
6.2.3	The data-source-kind-by-name Function	22
6.2.4	The data-source-kind-id-for Function	22
6.2.5	The all-data-source-kinds Function	22
6.2.6	The ensure-standard-data-source-kinds Function	22

7	The candlesticks/data-sources Package	23
7.1	Types	23
7.1.1	The data-source Type	23
7.2	Functions	23
7.2.1	The make-data-source Function	23
7.2.2	The data-source-by-name Function	24
7.2.3	The data-source-id-for Function	24
7.2.4	The all-data-sources Function	24
8	The candlesticks/exchanges Package	25
8.1	Types	25
8.1.1	The exchange Type	25
8.2	Functions	25
8.2.1	The standard-exchanges Function	25
8.2.2	The make-exchange Function	26
8.2.3	The exchange-by-name Function	26
8.2.4	The exchange-id-for Function	26
8.2.5	The lookup-exchange-id Function	26
8.2.6	The ensure-standard-exchanges Function	26
9	The candlesticks/tickers Package	27
9.1	Types	27
9.1.1	The ticker Type	27
9.2	Functions	28
9.2.1	The canonicalize-ticker Function	28
9.2.2	The make-ticker Function	28
9.2.3	The close-ticker Function	28
9.2.4	The ticker-by-source-and-symbol Function	28
9.2.5	The tickers-for-instrument Function	28
9.2.6	The tickers-for-symbol Function	29
9.2.7	The instrument-for-ticker Function	29
9.2.8	The ensure-instrument-for-ticker Function	29
10	The candlesticks/data-retrievals Package	31
10.1	Types	31
10.1.1	The data-retrieval Type	31
10.2	Functions	31
10.2.1	The record-retrieval Function	31
10.2.2	The retrieval-by-id Function	32
10.2.3	The data-retrieval-source Function	32
10.2.4	The data-retrieval-source-name Function	32
10.2.5	The retrievals-for-source Function	32

11 The candlesticks/bars Package	33
11.1 Types	33
11.1.1 The candlestick Type	33
11.2 Functions	33
11.2.1 The instrument-id-for Function	33
11.2.2 The duration-id-for Function	34
11.2.3 The insert-candlestick Function	34
11.2.4 The insert-candlesticks Function	34
11.2.5 The ingest-candlesticks Function	35
11.2.6 The get-candlesticks Function	35
11.2.7 The count-candlesticks Function	36
11.3 Projections	36
12 The candlesticks/migrations Package	37
12.1 Functions	37
12.1.1 The migrations-path Function	37
12.1.2 The connection-spec->migratum-spec Function	37
12.1.3 The run-migrations Function	38
12.1.4 The revert-last-migration Function	38
12.1.5 The pending-migrations Function	38
12.1.6 The applied-migrations Function	38
12.1.7 The latest-migration-id Function	38
12.1.8 The migration-status? Function	38
12.2 Macros	38
12.2.1 The with-migration-driver Macro	38
13 The candlesticks Package	41
13.1 Functions	41
13.1.1 The version-string Function	41
13.1.2 The version-list Function	41
Bibliography	43

Copyright

Copyright © 2026, Christopher Mark Gore,

Soli Deo Gloria,

All rights reserved.

22 Forest Glade Court, Saint Charles, Missouri 63304 USA.

Web: <http://www.cgore.com>

Email: cgore@cgore.com

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Christopher Mark Gore nor the names of other contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Introduction

Candlesticks is an interface to a PostgreSQL [6] database of historical, time-series financial data. It stores OHLC candlesticks — open, high, low, close, and an optional volume — for any fungible thing that trades in a market: stocks, bonds, cryptocurrencies, commodities, currencies, indices, and whatever else you point it at.

A price is always a pair of instruments. `BTC/USD` is Bitcoin as the numerator and the US Dollar as the denominator. `AAPL/USD` is Apple stock in dollars. The same tables hold all of them.

Every bar knows where it came from. A `data_sources` row names CoinGecko, Yahoo Finance, Zapper.fi, or whoever else produced the numbers. A `data_retrievals` row records one act of fetching a set of data from that source — when it happened, which endpoint was queried, and any notes. Each candlestick points at the retrieval that produced it. The same numbers from two sources, or from two retrievals of the same source, are two rows. That is how we keep multiple instances of the same data and still know where each one came from.

The long-term plan is for Livermore [2] to take its historical stock series from this library instead of from `livermore/source/stocks.lisp`, and for `https://limbic.fi` to store long-term cryptocurrency histories from `cl-coingecko` [1] the same way.

You can download the library from GitHub at:
<https://github.com/cgore/candlesticks>

The Schema

Tables, all with version-4 UUID primary keys generated by PostgreSQL's built-in `gen_random_uuid()` (PostgreSQL 13 or later). Schema changes are applied with `cl-migratum` [5]; queries go through Postmodern [4].

Table	What it holds
<code>instrument_types</code>	What an instrument is: stock, cryptocurrency, ...
<code>data_source_kinds</code>	How we talk to a source: api, website, feed, ...
<code>exchanges</code>	Where it traded: NASDAQ, Binance, Uniswap V3, ...
<code>instruments</code>	One instrument; identity is the UUID, not a ticker
<code>tickers</code>	Abbreviation a source (and optional exchange) uses for an instrument, with a validity window
<code>candlestick_durations</code>	Short name, long name, and a PostgreSQL interval
<code>data_sources</code>	Who told us: CoinGecko, Yahoo Finance, Zapper.fi, ...
<code>data_retrievals</code>	One fetch from a source, optionally pinned to an exchange
<code>candlesticks</code>	One OHLC bar, linked to a pair, a duration, and a retrieval

A data source is who told us. An exchange is where it traded — a stock exchange, a CEX, or a DEX (with an optional `chain`, so Uniswap V3 on Ethereum is not Uniswap V3 on Arbitrum). A market in the pair-on-a-venue sense is the bar's numerator and denominator plus the exchange; that is not a separate table yet.

A ticker is unique as (*data source, canonical abbreviation, exchange*) at a given time. CoinGecko's aggregated BTC and Yahoo's BTC on NYSE can be different instruments. Yahoo's GM before and after the 2009 bankruptcy can be different instruments too: each ticker row has a `valid_from / valid_to` window.

The unique key on `candlesticks` is (*numerator, denominator, duration, time, retrieval*). A check constraint forbids a bar whose numerator and denominator are the same instrument.

The conventional durations are second, minute, hour, day, week, month, and year, with short names `s`, `m`, `h`, `d`, `w`, `M`, and `y`. Sources that publish other windows can add their own rows.

Chapter 1 walks through every table, column, constraint, and index.

Getting Lisp

Before using this library you need a working Lisp. I use and recommend SBCL, Steel Bank Common Lisp, which is available at:

<http://www.sbcl.org>

SBCL has information on getting started at:

<http://www.sbcl.org/getting.html>

If you are using Debian or a similar Linux distribution (including Ubuntu), you can just run as root:

```
apt-get install sbcl sbcl-doc sbcl-source
```

You also need a PostgreSQL server, version 13 or later.

Getting EMACS and SLIME

After installing, the best way to interact with any Common Lisp is via SLIME, the Superior Lisp Interaction Mode for EMACS, which is available at:

<http://common-lisp.net/project/slime/>

This can be installed on Debian by:

```
apt-get install slime emacs emacs-goodies-el
```

Using the Library

Clone the repository next to your other local systems, or symlink the ASDF file into a directory on `asdf:*central-registry*` (I use `~/programming/lisp/systems/`).

```
mkdir -p ~/programming/lisp
cd ~/programming/lisp
git clone git@github.com:cgore/candlesticks.git
ln -s ~/programming/lisp/candlesticks/candlesticks.asd \
    ~/programmings/lisp/systems/
```

The system depends on `Σ` [3], Postmodern [4], `cl-migratum`, `cl-migratum.provider.local-path`, and `cl-migratum.driver.postmodern-postgresql`. Those come from Quicklisp (or from a local clone, which is how I load `cl-migratum`).

A connection spec is a list that `postmodern:connect` can be applied to, for example `("database" "user" "password" "host":port5432)`. `with-connection` opens one for the duration of its body. Nesting or repeating the form gives you several independent connections, which is how you work against more than one database at a time.

```
(ql:quickload :candlesticks)
(in-package :candlesticks)

(setf *default-connection-spec*
      '("candlesticks" "candlesticks" "secret" "localhost"
        :port 5432))

(with-connection nil
  (run-migrations *default-connection-spec*)
  (ensure-standard-durations))
```

```

(ensure-standard-instrument-types)
(ensure-standard-data-source-kinds)
(ingest-candlesticks "BTC" "USD" "d"
  (list (list (encode-universal-time 0 0
    0 18 8 2026 0)
    25000.0 26000.0 24000.0
    25500.0 100.0))
    :source-name "CoinGecko"
    :source-kind "api"
    :numerator-name "Bitcoin"
    :numerator-type "cryptocurrency"
    :denominator-name "US_Dollar"
    :denominator-type "currency")
(candlestick-close-prices (get-candlesticks "BTC" "USD"
  "d"))))

```

Times are Common Lisp universal times. Postmodern hands `times-tamptz` values back as universal times, and `numeric` values as rationals; the library converts the latter to floats on the way out.

Testing

Specs are `sigma/behavior behavior / should forms`, the same style as `Σ`. Pure helpers run at load time. `asdf:test-system` reloads the sources so those assertions run again.

```
(asdf:test-system :candlesticks)
```

To also hit a live PostgreSQL server, set two environment variables and run `asdf:test-system` again.

```

CANDLESTICKS_LIVE_TESTS=1
CANDLESTICKS_LIVE_DB="database_user_password_host_port"

```

The live suite creates a throwaway schema, runs the migrations, exercises the whole API — including provenance, duplicate ingestion, and nested connections — and drops the schema when it is done.

Have fun!

Chapter 1

The Schema

The database is ordinary PostgreSQL [6]. Schema changes live as numbered `.up.sql` / `.down.sql` pairs under `migrations/` and are applied with `cl-migratum` [5]. The initial (and so far only) migration is `20260818120000-create-schema`. Statements in those files are separated by `--;;` lines, which is how `cl-migratum` splits a file into steps.

This chapter is the map of that schema. The Lisp packages in the chapters that follow are a veneer over these tables.

1.1 Conventions

- Every table has a version-4 UUID primary key whose default is PostgreSQL's built-in `gen_random_uuid()`. That function lives in `pg_catalog`, so it is available from any schema without an extension (PostgreSQL 13 or later).
- Identity of an instrument is that UUID, not a ticker symbol. Abbreviations go in `tickers`.
- Timestamps are `timestamp with time zone` (`timestampz` below). The Lisp side speaks Common Lisp universal times.
- Prices, volume, and adjusted close are `numeric`. Postmodern hands those back as rationals; the library converts them to floats on the way out.
- A data source is who told us. An exchange is where it traded. A market in the pair-on-a-venue sense is not a table; it is the bar's numerator and denominator plus the exchange.
- Ticker validity windows are half-open, [*valid from*, *valid to*). A trigger forbids two overlapping windows for the same source, canon-

ical abbreviation, and exchange. The same abbreviation may be reused across non-overlapping eras.

1.2 How the Tables Fit Together

There are nine tables. Four of them are catalogs you seed once: instrument types, data source kinds, exchanges, and candlestick durations. Two of them name the things you care about: instruments and data sources. Tickers attach abbreviations. Retrievals and candlesticks are the growing facts: one fetch, then the bars that fetch produced.

- An instrument may have a type. A data source may have a kind.
- A ticker names an instrument at one data source, optionally on one exchange, during a validity window. The same abbreviation at two sources, or at two exchanges of one source, can be two instruments. The same abbreviation at one source can also mean different instruments in non-overlapping eras.
- A retrieval is one act of fetching from a source, optionally pinned to an exchange.
- A candlestick is the as-traded OHLC of a numerator instrument denominated in a denominator instrument, over one duration, at one time, from one retrieval. The unique key includes the retrieval, so two sources — or two fetches of the same source — are two rows.

Figure 1.1 is the same map drawn as foreign keys, with the candlesticks at the top and the catalogs they rest on cascading down. A solid arrow is a required key, from the table that holds the column to the table it references. A dashed arrow is nullable: a ticker or a retrieval may omit an exchange. A candlestick points at two instruments, the numerator and the denominator of the price.

Foreign keys:

- `instruments.instrument_type_id` → `instrument_types`
- `data_sources.data_source_kind_id` → `data_source_kinds`
- `tickers.instrument_id` → `instruments`
- `tickers.data_source_id` → `data_sources`
- `tickers.exchange_id` → `exchanges` (nullable)
- `data_retrievals.data_source_id` → `data_sources`
- `data_retrievals.exchange_id` → `exchanges` (nullable)

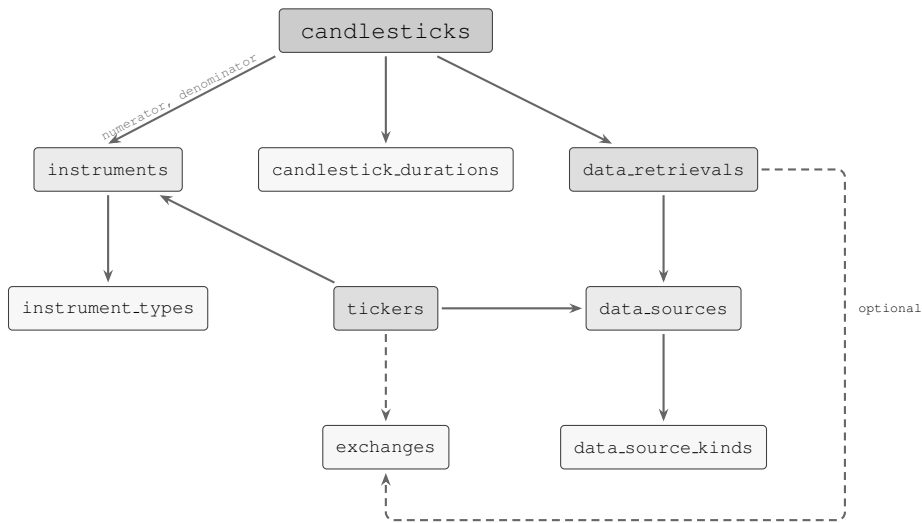


Figure 1.1: Foreign keys among the Candlesticks tables, hanging off the candlesticks themselves. Solid arrows are required; dashed arrows are nullable. Fill is darkest at the top and lightest on the catalogs at the bottom.

- `candlesticks.numerator_id` → `instruments`
- `candlesticks.denominator_id` → `instruments`
- `candlesticks.duration_id` → `candlestick_durations`
- `candlesticks.data_retrieval_id` → `data_retrievals`

1.3 Overview

Table	What it holds
<code>instrument_types</code>	What an instrument is: stock, cryptocurrency, ...
<code>data_source_kinds</code>	How we talk to a source: api, website, feed, ...
<code>exchanges</code>	Where it traded: NASDAQ, Binance, Uniswap V3, ...
<code>instruments</code>	One instrument; identity is the UUID, not a ticker
<code>data_sources</code>	Who told us: CoinGecko, Yahoo Finance, Zapper.fi, ...
<code>tickers</code>	Abbreviation a source (and optional exchange) uses, with a validity window
<code>candlestick_durations</code>	Short name, long name, and a PostgreSQL interval
<code>data_retrievals</code>	One fetch from a source, optionally pinned to an exchange
<code>candlesticks</code>	One OHLC bar, linked to a pair, a duration, and a retrieval

The Lisp packages that talk to each table:

Table	Package
<code>instrument_types</code>	<code>candlesticks/instrument-types</code>
<code>data_source_kinds</code>	<code>candlesticks/data-source-kinds</code>
<code>exchanges</code>	<code>candlesticks/exchanges</code>
<code>instruments</code>	<code>candlesticks/instruments</code>
<code>data_sources</code>	<code>candlesticks/data-sources</code>
<code>tickers</code>	<code>candlesticks/tickers</code>
<code>candlestick_durations</code>	<code>candlesticks/durations</code>
<code>data_retrievals</code>	<code>candlesticks/data-retrievals</code>
<code>candlesticks</code>	<code>candlesticks/bars</code>

Migrations live in `candlesticks/migrations`. Connections live in `candlesticks/config`. The umbrella package `candlesticks` re-exports the rest.

1.4 The `instrument_types` Table

What an instrument *is*: stock, bond, cryptocurrency, and so on. This is a different ontology from data source kinds, which classify *how we talk to a source*.

Names are unique. The Lisp side trims and down-cases them with `canonicalize-slug`, so "Cryptocurrency" and " cryptocurrency

" are the same type.

Column	Type	Notes
id	uuid	Primary key
name	varchar	Not null, unique
description	varchar	Optional
created_at	timestampz	Not null, default now()

The conventional seed, from ensure-standard-instrument-types:

Name	Description
stock	Equity listed on an exchange
bond	Debt instrument
commodity	Fungible physical or futures-traded good
currency	Fiat currency
cryptocurrency	Native on-chain asset
index	Published basket or benchmark, not itself traded
fund	Pooled vehicle (mutual fund, ETF, ...)

1.5 The data_source_kinds Table

How Candlesticks talks to a source: api, website, feed, file, or manual. A source row is “the CoinGecko API we call”, not a catalog of everything CoinGecko Inc. does, so one kind per source is enough.

Names are unique and canonicalized the same way as instrument types.

Column	Type	Notes
id	uuid	Primary key
name	varchar	Not null, unique
description	varchar	Optional
created_at	timestampz	Not null, default now()

The conventional seed, from ensure-standard-data-source-kinds:

Name	Description
api	Programmatic HTTP/RPC API
website	HTML or other page we scrape or parse
feed	Push feed (websocket, FIX, ...)
file	File or dump (Csv, JSON, ...)
manual	Entered or corrected by hand

1.6 The exchanges Table

Where instruments trade: a stock exchange, a CEX, or a DEX. That is not a data source, and not a market in the pair-on-a-venue sense.

The same protocol on two chains is two exchanges. Uniswap V3 on Ethereum and Uniswap V3 on Arbitrum are different rows. Traditional venues may carry an ISO 10383 MIC: the four-character Market Identifier Code from the official registry (XNAS for NASDAQ, XNYS for NYSE). The column is optional and unique when present. Crypto venues and DEXes usually leave it null and use `chain` instead.

Column	Type	Notes
<code>id</code>	<code>uuid</code>	Primary key
<code>name</code>	<code>varchar</code>	Canonical slug, not null, unique
<code>display_name</code>	<code>varchar</code>	Optional human label
<code>mic</code>	<code>varchar</code>	Optional ISO 10383 Market Identifier Code; unique when present
<code>chain</code>	<code>varchar</code>	Optional; indexed
<code>description</code>	<code>varchar</code>	Optional
<code>created_at</code>	<code>timestamp tz</code>	Not null, default <code>now()</code>

Index: `exchanges_chain_index` on `chain`.

The conventional seed, from `ensure-standard-exchanges`:

Name	Display name	Venue
<code>nasdaq</code>	NASDAQ	MIC XNAS
<code>nyse</code>	New York Stock Exchange	MIC XNYS
<code>binance</code>	Binance	centralized crypto
<code>coinbase</code>	Coinbase	centralized crypto
<code>kraken</code>	Kraken	centralized crypto
<code>uniswap-v3-ethereum</code>	Uniswap V3 (Ethereum)	<code>chain</code> <code>ethereum</code>
<code>uniswap-v3-arbitrum</code>	Uniswap V3 (Arbitrum)	<code>chain</code> <code>arbitrum</code>
<code>uniswap-v2-ethereum</code>	Uniswap V2 (Ethereum)	<code>chain</code> <code>ethereum</code>
<code>sushiswap-ethereum</code>	SushiSwap (Ethereum)	<code>chain</code> <code>ethereum</code>
<code>curve-ethereum</code>	Curve (Ethereum)	<code>chain</code> <code>ethereum</code>
<code>pancakeswap-bsc</code>	PancakeSwap (BSC)	<code>chain</code> <code>bsc</code>
<code>orca-solana</code>	Orca (Solana)	<code>chain</code> <code>solana</code>
<code>raydium-solana</code>	Raydium (Solana)	<code>chain</code> <code>solana</code>

Other venues are created on first use.

1.7 The instruments Table

One tradable thing: a stock, a coin, a currency, whatever. Identity is the UUID. `name` is a human label such as "Bitcoin" and is neither unique nor required. Short market abbreviations live on `tickers`,

because the same abbreviation can mean different instruments at different sources.

Column	Type	Notes
id	uuid	Primary key
name	varchar	Optional display name, not unique
instrument_type_id	uuid	Optional FK to instrument_types
created_at	timestampz	Not null, default now()
updated_at	timestampz	Not null, default now()

Index: instruments_type_index on instrument_type_id.

1.8 The data_sources Table

Whoever actually produced the numbers: CoinGecko, Zapper.fi, Yahoo Finance, a Csv dump, a proprietary feed. Retrievals point at a source, and candlesticks point at a retrieval, so every bar can be traced all the way down.

Column	Type	Notes
id	uuid	Primary key
name	varchar	Not null, unique
data_source_kind_id	uuid	Optional FK to data_source_kinds
created_at	timestampz	Not null, default now()

Index: data_sources_kind_index on data_source_kind_id.

There is no seeded catalog of sources. A source row is created the first time you ingest from it.

1.9 The tickers Table

The abbreviation a data source uses for an instrument, optionally on a particular exchange. It is *not* the identity of the instrument. CoinGecko's aggregated BTC (no exchange) can be Bitcoin while Yahoo's BTC on NYSE is a different instrument. Yahoo's AAPL and Bloomberg's AAPL can both point at the same Apple row.

The same (*data source, canonical symbol, exchange*) may appear more than once, so long as the validity windows do not overlap. A missing exchange still counts: two aggregator tickers for the same symbol at the same source cannot be valid at the same moment. The window is half-open, [*valid from*, *valid to*). An unbounded start is stored as `-infinity`; a null `valid to` means the abbreviation is still in use.

`symbol` is what the source called it. `canonical` is that abbreviation trimmed and upper-cased, so "btc" and " BTC " match *within one source*.

Column	Type	Notes
<code>id</code>	<code>uuid</code>	Primary key
<code>instrument_id</code>	<code>uuid</code>	Not null, FK to <code>instruments</code>
<code>data_source_id</code>	<code>uuid</code>	Not null, FK to <code>data-sources</code>
<code>exchange_id</code>	<code>uuid</code>	Optional FK to <code>exchanges</code>
<code>symbol</code>	<code>varchar</code>	As the source wrote it, not null
<code>canonical</code>	<code>varchar</code>	Normalized form, not null
<code>valid_from</code>	<code>timestampz</code>	Not null, default -infinity
<code>valid_to</code>	<code>timestampz</code>	Optional; null means still valid
<code>created_at</code>	<code>timestampz</code>	Not null, default <code>now()</code>

Indexes:

- `tickers.instrument_index` on `instrument_id`
- `tickers.canonical_index` on `canonical`
- `tickers.exchange_index` on `exchange_id`
- `tickers.lookup_index` on `source`, `canonical`, and `valid_from`

A trigger forbids two overlapping windows for the same source, canonical abbreviation, and exchange. A check constraint requires `valid_to` to be after `valid_from` when both are present.

1.10 The candlestick_durations Table

One candlestick time window: a short name (such as "w"), a long name (such as "week"), and the PostgreSQL interval that corresponds to it.

Short names are unique on their own, and so are long names — there is only one "d" and only one "week". That is two unique constraints, not one unique pair.

Column	Type	Notes
<code>id</code>	<code>uuid</code>	Primary key
<code>short_name</code>	<code>varchar</code>	Not null, unique
<code>long_name</code>	<code>varchar</code>	Not null, unique
<code>duration</code>	<code>interval</code>	Not null

The conventional seed, from ensure-standard-durations:

Short	Long	Interval
s	second	1 second
m	minute	1 minute
h	hour	1 hour
d	day	1 day
w	week	1 week
M	month	1 month
y	year	1 year

Sources that publish other windows can add their own rows. Note that month is M (upper case) so it does not collide with minute.

1.11 The data_retrievals Table

One act of fetching a set of data from a data source: when it happened, the endpoint that was queried, and any notes. Every candlestick points back to the retrieval that produced it. An optional `exchange_id` pins the fetch to a venue when the source was asked about one.

Column	Type	Notes
id	uuid	Primary key
data_source_id	uuid	Not null, FK to data-sources
exchange_id	uuid	Optional FK to exchanges
retrieved_at	timestampz	Not null, default now()
endpoint	varchar	Optional
notes	text	Optional
created_at	timestampz	Not null, default now()

Indexes:

- data_retrievals_source_index on data_source_id
- data_retrievals_exchange_index on exchange_id
- data_retrievals_retrieved_index on retrieved_at

1.12 The candlesticks Table

One OHLC bar. A price is always a pair of instruments: BTC/USD is Bitcoin as the numerator and the US Dollar as the denominator. Open, high, low, and close are as-traded. `adjusted_close` is the source's split-adjusted close when it reported one, and may be null. Volume may be null.

The unique key is (*numerator*, *denominator*, *duration*, *time*, *retrieval*). Inserting the same bar twice against one retrieval is a no-op; ingesting it again under a new retrieval stores a second instance.

Column	Type	Notes
id	uuid	Primary key
numerator_id	uuid	Not null, FK to instruments
denominator_id	uuid	Not null, FK to instruments
duration_id	uuid	Not null, FK to candlestick_durations
data_retrieval_id	uuid	Not null, FK to data_retrievals
time	timestampz	Not null; the bar's period
open	numeric	Not null, as-traded
high	numeric	Not null, as-traded
low	numeric	Not null, as-traded
close	numeric	Not null, as-traded
volume	numeric	Optional
adjusted_close	numeric	Optional split-adjusted close
created_at	timestampz	Not null, default now()

Check constraints:

- The numerator and the denominator are different instruments.
- $\text{high} \geq \text{low}$.
- high is at least open and at least close.
- low is at most open and at most close.
- volume is null or ≥ 0 .
- adjusted_close is null or ≥ 0 .

Indexes:

- candlesticks_query_index on numerator, denominator, duration, and time — the lookup get-candlesticks uses
- candlesticks_retrieval_index on data_retrieval_id

1.13 What Is Not a Table

A market in the pair-on-an-exchange sense is the candlestick's numerator and denominator plus the exchange. That can wait; it is not a separate relation yet.

Chapter 2

The candlesticks/config Package

The `candlesticks/config` package holds the connection helpers and the small conversions every other package needs. A connection spec is a list that `postmodern:connect` can be applied to, for example `("database" "user" "password" "host":port5432)`.

2.1 Variables

2.1.1 The `*default-connection-spec*` Variable

The spec `with-connection` uses when it is not given one. Initially `nil`. Set it with `set-default-connection-spec!`, or bind it around a call.

2.2 Functions

2.2.1 The `default-connection-spec` Function

The current value of `*default-connection-spec*`.

2.2.2 The `set-default-connection-spec!` Function

Set `*default-connection-spec*` to *spec-list* and return it.

Syntax

```
(set-default-connection-spec! spec-list)
```

2.2.3 The connection-spec Function

Return *spec-list*, or **default-connection-spec** when *spec-list* is *nil*. Signals an error when neither is available.

Syntax

```
(connection-spec &optional spec-list)
```

2.2.4 The current-connection Function

The active Postmodern connection, or *nil* when none is bound. This is *postmodern:database**.

2.2.5 The connected-p Function

True if *connection* (default: the active connection) is open and connected.

Syntax

```
(connected-p &optional connection)
```

2.2.6 The require-connection! Function

Return the active Postmodern connection, signalling a clear error if there is none.

2.2.7 The universal-time->timestampz Function

Format a universal time as a PostgreSQL *timestampz* literal in UTC, e.g. "2026-08-18 12:00:00+00". This is the form Postmodern and PostgreSQL accept as a parameter. When such a value is read back, Postmodern hands it to us as a universal time again.

Syntax

```
(universal-time->timestampz universal-time)
```

2.2.8 The canonicalize-slug Function

Trim leading and trailing whitespace from a name and down-case it, so that "API", " api ", and "Api" are the same slug. Used for instrument type names and data source kind names.

Syntax

```
(canonicalize-slug name)
```

2.2.9 The to-db Function

Convert a Lisp `nil` into the Postmodern NULL marker `:null`. Any other value is returned unchanged.

2.2.10 The from-db Function

Convert a Postmodern `:null` into Lisp `nil`. Any other value is returned unchanged.

2.2.11 The coerce-price Function

Coerce a numeric value read back from the database (Postmodern returns numeric columns as rationals) into a float. `nil` and `:null` pass through as `nil`, so nullable columns such as volume stay `nil`.

2.3 Macros**2.3.1 The with-connection Macro**

Evaluate *body* with the active database connection bound to a connection opened from *spec-list*. When *spec-list* is `nil`, `*default-connection-spec*` is used instead. The connection is closed when *body* finishes. Nesting or repeating the form gives you several independent connections.

Syntax

```
(with-connection spec-list &body body)
```

Examples

```
(with-connection '("db" "user" "pw" "localhost" :port 5432)
  (all-instruments))

(let ((*default-connection-spec* '("db" "user" "pw"
  "localhost" :port 5432)))
  (with-connection nil
    (all-instruments)))
```

2.3.2 The first-row Macro

Run a parameterized SQL statement and return the first row as a list of values, or `nil` when there is no row. This is a macro so that its parameters are compile-time forms, which is what Postmodern's `query` macro requires.

Syntax

```
(first-row sql &rest params)
```

Chapter 3

The candlesticks/ durations Package

A duration is one candlestick time window: a short name (such as "w"), a long name (such as "week"), and the PostgreSQL interval that corresponds to it. The conventional set is second, minute, hour, day, week, month, and year.

3.1 Types

3.1.1 The duration Type

A CLOS class with slots `id`, `short-name`, `long-name`, and `interval`. The accessors are `duration-id`, `duration-short-name`, `duration-long-name`, and `duration-interval`. `durationp` is true of instances.

3.2 Functions

3.2.1 The standard-durations Function

The conventional `(short-name long-name interval)` triples. A fresh copy is returned, so callers may mutate the result freely.

3.2.2 The make-duration Function

Ensure a duration with the given short name, long name, and PostgreSQL interval exists, and return it. Short names are unique on their own, and so are long names — there is only one "d" and only one "week". If those two names already belong to different rows, that is an error.

Syntax

```
(make-duration short-name long-name interval)
```

3.2.3 The duration-by-short-name Function

The duration with the given short name, or `nil`.

3.2.4 The duration-by-long-name Function

The duration with the given long name, or `nil`.

3.2.5 The all-durations Function

Every duration, in short-name order.

3.2.6 The ensure-standard-durations Function

Insert the standard second / minute / hour / day / week / month / year durations if they are not already present, and return them. Call this once after the migrations have been applied.

Examples

```
(ensure-standard-durations)
(duration-short-name (duration-by-long-name "week"))
; => "W"
```

Chapter 4

The candlesticks/ instrument-types Package

An instrument type is a classification of *what an instrument is*: stock, bond, cryptocurrency, commodity, currency, index, fund. This is a different ontology from data source kinds, which classify *how we talk to a source*.

Names are canonicalized by `canonicalize-slug`: trimmed and down-cased, so "Cryptocurrency" and " cryptocurrency " are the same type.

4.1 Types

4.1.1 The instrument-type Type

A CLOS class with slots `id`, `name`, and `description`. The accessors are `instrument-type-id`, `instrument-type-name`, and `instrument-type-description`. `instrument-type-p` is true of instances.

4.2 Functions

4.2.1 The standard-instrument-types Function

The conventional `(name description)` pairs. A fresh copy is returned, so callers may mutate the result freely.

The seeded names are `stock`, `bond`, `commodity`, `currency`, `cryptocurrency`, `index`, and `fund`.

4.2.2 The make-instrument-type Function

Find or create the instrument type with the given name and return it. `:description`, when given, fills in a missing description on an existing type.

Syntax

```
(make-instrument-type name &key description)
```

4.2.3 The instrument-type-by-name Function

The instrument type whose canonical name is *name*, or `nil`. This does not create a row.

4.2.4 The instrument-type-id-for Function

The database UUID of *type*, or `nil` when *type* is `nil`. *type* may be an `instrument-type` object or a name string (which is found or created).

4.2.5 The all-instrument-types Function

Every instrument type, alphabetically by name.

4.2.6 The ensure-standard-instrument-types Function

Insert the standard types if they are not already present, and return them. Call this once after the migrations have been applied.

Chapter 5

The candlesticks/ instruments Package

An instrument is one tradable thing: a stock, bond, cryptocurrency, commodity, currency, index, or anything else that exists in a fungible traded market. Identity is the UUID. Short market abbreviations are `tickers`, because the same abbreviation can mean different instruments at different sources.

5.1 Types

5.1.1 The instrument Type

A CLOS class with slots `id`, `name`, `instrument-type-id`, and `instrument-type`. The accessors have the obvious names. `instrumentp` is true of instances. `name` is a human label such as "Bitcoin" and is not unique.

5.2 Functions

5.2.1 The `make-instrument` Function

Create a new instrument and return it. Identity is the UUID; attach market abbreviations with `make-ticker` or `ensure-instrument-for-ticker`.

Syntax

```
(make-instrument &key name instrument-type)
```

5.2.2 The `fill-instrument` Function

Fill in any missing name or type on an existing instrument, without overwriting values that are already set.

5.2.3 The `instrument-by-id` Function

The instrument with the given UUID, or `nil`.

5.2.4 The `all-instruments` Function

Every instrument, alphabetically by name.

Chapter 6

The candlesticks/data-source-kinds Package

A data source kind is a classification of *how Candlesticks talks to a source*: api, website, feed, file, or manual. A source row is “the CoinGecko API we call”, not a catalog of everything CoinGecko Inc. does, so one kind per source is enough.

Names are canonicalized by `canonicalize-slug`.

6.1 Types

6.1.1 The data-source-kind Type

A CLOS class with slots `id`, `name`, and `description`. The accessors are `data-source-kind-id`, `data-source-kind-name`, and `data-source-kind-description`. `data-source-kind-p` is true of instances.

6.2 Functions

6.2.1 The standard-data-source-kinds Function

The conventional `(name description)` pairs. A fresh copy is returned, so callers may mutate the result freely.

The seeded names are `api`, `website`, `feed`, `file`, and `manual`.

6.2.2 The make-data-source-kind Function

Find or create the data source kind with the given name and return it. `:description`, when given, fills in a missing description on an existing kind.

Syntax

`(make-data-source-kind name &key description)`

6.2.3 The data-source-kind-by-name Function

The data source kind whose canonical name is *name*, or *nil*. This does not create a row.

6.2.4 The data-source-kind-id-for Function

The database UUID of *kind*, or *nil* when *kind* is *nil*. *kind* may be a `data-source-kind` object or a name string (which is found or created).

6.2.5 The all-data-source-kinds Function

Every data source kind, alphabetically by name.

6.2.6 The ensure-standard-data-source-kinds Function

Insert the standard kinds if they are not already present, and return them. Call this once after the migrations have been applied.

Chapter 7

The candlesticks/data-sources Package

A data source is whoever actually produced the numbers: CoinGecko, Zapper.fi, Yahoo Finance, a CSV dump, a proprietary feed. Retrievals point at a source, and candlesticks point at a retrieval, so every bar can be traced all the way down.

7.1 Types

7.1.1 The data-source Type

A CLOS class with slots `id`, `name`, `data-source-kind-id`, and `data-source-kind`. The accessors have the obvious names. `data-source-kind` is the related `data-source-kind` object, or `nil`. `data-source-kind-name` on a source is the kind's canonical name, or `nil`.

7.2 Functions

7.2.1 The make-data-source Function

Find or create the data source with the given name and return it. `:kind` is a `data-source-kind` or a name such as `"api"`; when given, it fills in a missing kind on an existing source.

Syntax

```
(make-data-source name &key kind)
```

7.2.2 The data-source-by-name Function

The data source with the given name, or `nil`. This does not create a row.

7.2.3 The data-source-id-for Function

The database UUID of the data source called *name*, created if it does not exist yet.

Syntax

```
(data-source-id-for name &optional kind)
```

7.2.4 The all-data-sources Function

Every data source, alphabetically by name.

Chapter 8

The candlesticks/ exchanges Package

An exchange is a *venue*: where instruments trade. That is not a data source (who told us) and not a market in the pair-on-a-venue sense (that is the candlestick's numerator and denominator plus this exchange).

The same protocol on two chains is two exchanges: `uniswap-v3-ethereum` vs `uniswap-v3-arbitrum`. Traditional venues may carry an ISO 10383 MIC: the four-character Market Identifier Code from the official registry (`XNAS` for NASDAQ, `XYNY` for NYSE). It is optional and unique when present; the accessor is `exchange-mic`. Crypto venues and DEXes usually leave it `nil` and use `chain` instead.

8.1 Types

8.1.1 The exchange Type

A CLOS class with slots `id`, `name`, `display-name`, `mic`, `chain`, and `description`. The accessors have the obvious names. `mic` is an ISO 10383 Market Identifier Code, or `nil`. `exchange-p` is true of instances.

8.2 Functions

8.2.1 The standard-exchanges Function

The conventional seed list: NASDAQ, NYSE, Binance, Coinbase, Kraken, and a handful of DEXes including Uniswap V3 on Ethereum and Arbitrum. Other venues are created on first use.

8.2.2 The make-exchange Function

Find or create the exchange with the given name (a canonical slug) and return it.

Syntax

```
(make-exchange name &key display-name mic chain description)
```

8.2.3 The exchange-by-name Function

The exchange whose canonical name is *name*, or *nil*.

8.2.4 The exchange-id-for Function

The database UUID of *exchange*, or *nil* when *exchange* is *nil*. A name string is found or created.

8.2.5 The lookup-exchange-id Function

Like *exchange-id-for*, but does not create a row.

8.2.6 The ensure-standard-exchanges Function

Insert the standard venues if they are not already present.

Chapter 9

The candlesticks/ tickers Package

A ticker is the abbreviation a data source uses for an instrument, optionally on a particular exchange. It is *not* the identity of the instrument. CoinGecko's aggregated BTC (no exchange) can be Bitcoin while Yahoo's BTC on NYSE is a different instrument. Yahoo's AAPL and Bloomberg's AAPL can both point at the same Apple row.

Uniqueness is (*data source, canonical symbol, exchange*) at a given time. Validity is the half-open window [*valid from, valid to*). *exchange* is `nil` when the source does not pin a venue (an aggregator). The same abbreviation can be reused for a different instrument after the previous window closes — Yahoo's GM before and after the 2009 bankruptcy, for example.

9.1 Types

9.1.1 The ticker Type

A CLOS class with slots `id`, `instrument-id`, `source-id`, `exchange-id`, `symbol`, `canonical`, `valid-from`, and `valid-to`. The accessors have the obvious names. `valid-from` and `valid-to` are universal times, or `nil` for unbounded (the beginning, or still valid). `tickerp` is true of instances.

9.2 Functions

9.2.1 The canonicalize-ticker Function

Trim and upper-case a ticker symbol so that "btc" and " BTC " match *within one source*.

Examples

```
(canonicalize-ticker "__btc__") ; => "BTC"
```

9.2.2 The make-ticker Function

Attach *symbol* as a ticker for *instrument* at *source*, optionally on *exchange*, during [*valid-from*, *valid-to*). If that triple is already valid at *at* (default: now, or *valid-from* when that is given), the existing ticker is returned and is not reassigned to a different instrument. To reuse the abbreviation for a new instrument, *close-ticker* the old row first.

Syntax

```
(make-ticker instrument symbol &key source exchange valid-from
 valid-to at)
```

9.2.3 The close-ticker Function

Set the ticker's *valid-to* to *at* (default: now). After that the abbreviation can be attached to a different instrument from that moment onward.

Syntax

```
(close-ticker ticker &optional at)
```

9.2.4 The ticker-by-source-and-symbol Function

The ticker for *symbol* at *source* valid at *at* (default: now), or nil. Does not create a row.

9.2.5 The tickers-for-instrument Function

Every ticker attached to an instrument, including expired windows.

9.2.6 The `tickers-for-symbol` Function

Every ticker whose canonical abbreviation is *symbol*, across all sources and validity windows.

9.2.7 The `instrument-for-ticker` Function

The instrument that *source* calls *symbol* at *at* (default: `now`), or `nil`. When *source* is omitted, the instrument is returned only if every matching ticker valid at that time points at the same instrument; otherwise `nil`, so an ambiguous abbreviation is not guessed. Does not create a row.

9.2.8 The `ensure-instrument-for-ticker` Function

Find the instrument that *source* calls *symbol* at *at* (default: `now`), or create both the instrument and the ticker. This is what `ingest` uses. To alias another source onto an instrument you already have, call `make-ticker` with that instrument instead.

Syntax

```
(ensure-instrument-for-ticker symbol &key source exchange name  
  instrument-type at valid-from valid-to)
```


Chapter 10

The candlesticks/data-retrievals Package

A data retrieval is one act of fetching a set of data from a data source: when it happened, the endpoint that was queried, and any notes. Every candlestick points back to the retrieval that produced it.

10.1 Types

10.1.1 The data-retrieval Type

A CLOS class with slots `id`, `source-id`, `retrieved-at`, `endpoint`, and `notes`. The accessors are `data-retrieval-id`, `data-retrieval-source-id`, `data-retrieval-retrieved-at`, `data-retrieval-endpoint`, and `data-retrieval-notes`. `retrieved-at` is a universal time.

10.2 Functions

10.2.1 The record-retrieval Function

Record that a set of data was retrieved from the data source with the given *source-id*, and return the new retrieval. `:retrieved-at` is a universal time (default: `now`). `:endpoint` and `:notes` are optional provenance details.

Syntax

```
(record-retrieval source-id &key retrieved-at endpoint notes)
```

10.2.2 The retrieval-by-id Function

The retrieval with the given UUID, or `nil`.

10.2.3 The data-retrieval-source Function

The `data-source` that the retrieval came from.

10.2.4 The data-retrieval-source-name Function

The name of the data source that the retrieval came from.

10.2.5 The retrievals-for-source Function

The most recent retrievals from the data source with the given `source-id`, newest first. `:limit` defaults to 100.

Syntax

```
(retrievals-for-source source-id &key limit)
```

Chapter 11

The candlesticks/bars Package

This is the package that reads and writes the candlesticks themselves. A candlestick is the price of a numerator instrument denominated in a denominator instrument, over a given duration, together with the retrieval that produced it. For example, a BTC/USD daily bar has Bitcoin as its numerator, the US Dollar as its denominator, and the day duration.

The unique key includes the retrieval, so the same numbers from two sources — or from two retrievals of the same source — are two rows.

11.1 Types

11.1.1 The candlestick Type

A CLOS class with slots `id`, `numerator-id`, `denominator-id`, `duration-id`, `retrieval-id`, `time`, `open`, `high`, `low`, `close`, `volume`, and `adjusted-close`. The accessors have the obvious names. `time` is a universal time. `volume` and `adjusted-close` may be `nil`. `open/high/low/close` are `as-traded`; `adjusted-close` is the source's split-adjusted close when it reported one.

11.2 Functions

11.2.1 The `instrument-id-for` Function

Return the database UUID of *instrument*. *instrument* may be an `instrument` object, or a ticker string resolved (and created if needed)

at :source.

Syntax

```
(instrument-id-for instrument &key name instrument-type source
exchange at)
```

11.2.2 The duration-id-for Function

Return the database UUID of *duration*. *duration* may be a duration object, a short name ("d", "w", ...), a long name ("day", "week", ...), or an existing duration UUID.

11.2.3 The insert-candlestick Function

Insert a single candlestick given its already-resolved identifiers and return the new row. *time* is a universal time; the price fields are numbers; *volume* may be nil. A conflicting (*numerator*, *denominator*, *duration*, *time*, *retrieval*) pair is a no-op.

Syntax

```
(insert-candlestick numerator-id denominator-id duration-id
retrieval-id time open high low close &optional volume
adjusted-close)
```

11.2.4 The insert-candlesticks Function

Insert a batch of candlestick rows, each a (time open high low close volume) or (time open high low close volume adjusted-close) tuple, and return two values: the number of rows inserted, and the retrieval that records the batch's provenance (or nil when :retrieval-id was supplied).

numerator and *denominator* are instrument symbols or instrument objects; *duration* is a duration (see duration-id-for); :source-name names the data source. When :retrieval-id is supplied, the bars are attached to that existing retrieval instead of creating a new one.

Syntax

```
(insert-candlesticks rows &key numerator denominator duration
retrieval-id source-name source-kind endpoint notes
retrieved-at)
```

11.2.5 The ingest-candlesticks Function

The high-level entry point for feeding data in from a source such as CoinGecko or Yahoo Finance. Store a batch of OHLC candlesticks in a single transaction and return two values: the number of rows inserted, and the retrieval that records where they came from.

Each call records a new retrieval, so ingesting the same numbers again stores a second instance of the data. Inserting the same bars twice against one existing retrieval is a no-op.

Syntax

```
(ingest-candlesticks numerator denominator duration bars &key
  source-name source-kind exchange endpoint notes numerator-name
  numerator-type denominator-name denominator-type retrieved-at
  at)
```

Arguments and Values

numerator, denominator Instrument symbols or instrument objects.

duration A short name, long name, duration, or id.

bars A list of (time open high low close volume) or (time open high low close volume adjusted-close) tuples. *time* is a universal time; *volume* and *adjusted-close* may be nil.

source-name The name of the data source, created if needed.

Examples

```
(ingest-candlesticks "BTC" "USD" "d"
  (list (list (encode-universal-time 0 0 0 18 8 2026 0)
    25000.0 26000.0 24000.0 25500.0 100.0))
  :source-name "CoinGecko" :source-kind "api"
  :numerator-name "Bitcoin" :numerator-type "cryptocurrency"
  :denominator-name "USDollar" :denominator-type "currency")
```

11.2.6 The get-candlesticks Function

Return the candlesticks for *numerator/denominator* at the given duration, in time-ascending order. *:from* and *:to*, when given, are universal times bounding the bar time (inclusive). *:source*, when given, restricts the result to bars retrieved from that data source.

This function never inserts rows. If the pair, duration, or source does not already exist, the result is the empty list.

Syntax

*(get-candlesticks numerator denominator duration &key from to
source exchange at)*

11.2.7 The count-candlesticks Function

The number of stored candlesticks for the pair at the given duration, summed over all retrievals. Returns 0 when the pair or duration is not already present; this function never inserts rows.

11.3 Projections

These take a list of candlesticks and return a list of one field, in the same order. `candlestick-close-prices` is the series most often handed to technical indicators. `close-series` returns `(time close)` pairs, a convenient shape for feeding straight into a time series.

- `candlestick-times`
- `candlestick-open-prices`
- `candlestick-high-prices`
- `candlestick-low-prices`
- `candlestick-close-prices`
- `candlestick-volumes`
- `candlestick-adjusted-close-prices`
- `close-series` — `as-traded` `(time close)` pairs
- `adjusted-close-series` — `split-adjusted` `(time adjusted-close)` pairs

Chapter 12

The candlesticks/ migrations Package

Schema changes live as numbered `.up.sql / .down.sql` pairs under `migrations/` and are applied with `cl-migratum` [5]. The Postmodern driver takes ownership of Postmodern's top-level connection, so every public function here opens that connection, does its work, and disconnects.

An optional schema name sets the connection's `search_path` first, which is how the live tests keep their tables out of `public`.

12.1 Functions

12.1.1 The `migrations-path` Function

The directory holding the Candlesticks migration files.

12.1.2 The `connection-spec->migratum-spec` Function

Convert a Postmodern connection spec (a list that `postmodern:connect` can be applied to) into the keyword spec the `cl-migratum` Postmodern driver expects.

Syntax

```
(connection-spec->migratum-spec spec-list)
```

Examples

```
(connection-spec->migratum-spec
  '("db" "u" "pw" "localhost" :port 5432))
; => (:database "db" :user-name "u" :password "pw"
;      :host "localhost" :port 5432)
```

12.1.3 The run-migrations Function

Apply all pending Candlesticks migrations to the database described by *spec-list*. When *schema* is given, it is first set as the connection's *search-path*.

Syntax

```
(run-migrations spec-list &optional schema)
```

12.1.4 The revert-last-migration Function

Revert the most recently applied Candlesticks migration.

12.1.5 The pending-migrations Function

The Candlesticks migrations that have not yet been applied.

12.1.6 The applied-migrations Function

The Candlesticks migrations that have already been applied.

12.1.7 The latest-migration-id Function

The id of the most recently applied Candlesticks migration, or *nil*.

12.1.8 The migration-status? Function

True if at least one Candlesticks migration has been applied to the given database (in *schema*, when given).

12.2 Macros

12.2.1 The with-migration-driver Macro

Create a *cl-migratum* driver for *spec-list*, optionally set the connection's *search-path* to *schema*, initialize the driver, run *body* with it bound to *driver-var*, and disconnect the top-level connection when *body* finishes.

Syntax

```
(with-migration-driver (driver-var &optional spec-list schema)  
  &body body)
```


Chapter 13

The candlesticks Package

The umbrella package, defined with `uiop:define-package` and `:reexport`. Every public symbol from the sub-packages — including the version helpers from `candlesticks/system` — is used and re-exported, so that `(use-package :candlesticks)` — or simply `(in-package :candlesticks)` — gives you the whole library at once. New exports on a sub-package show up here without editing this file.

13.1 Functions

13.1.1 The version-string Function

The library version as a dotted string, currently "0.2.0".

13.1.2 The version-list Function

The library version as a list of three integers, (*major minor revision*).

The parameters `version-major`, `version-minor`, and `version-revision` hold the three pieces separately.

Bibliography

- [1] GORE, C. M. `cl-coingecko`. <https://github.com/cgore/cl-coingecko>, 2026.
- [2] GORE, C. M. `Livermore`. <https://github.com/cgore/livermore>, 2026.
- [3] GORE, C. M. Σ , a Library for ANSI Common Lisp. <https://github.com/cgore/sigma>, 2026.
- [4] HAVERBEKE, M., AND CROLLETON, S. `Postmodern`. <https://github.com/marijnh/Postmodern>, 2024.
- [5] NIKOLOV, M. A. `cl-migratum`. <https://github.com/dnaeon/cl-migratum>, 2022.
- [6] THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation. <https://www.postgresql.org/docs/>, 2026.